

Assignment - 17.1

Name :B.Sarayu

H.No:2303A51842

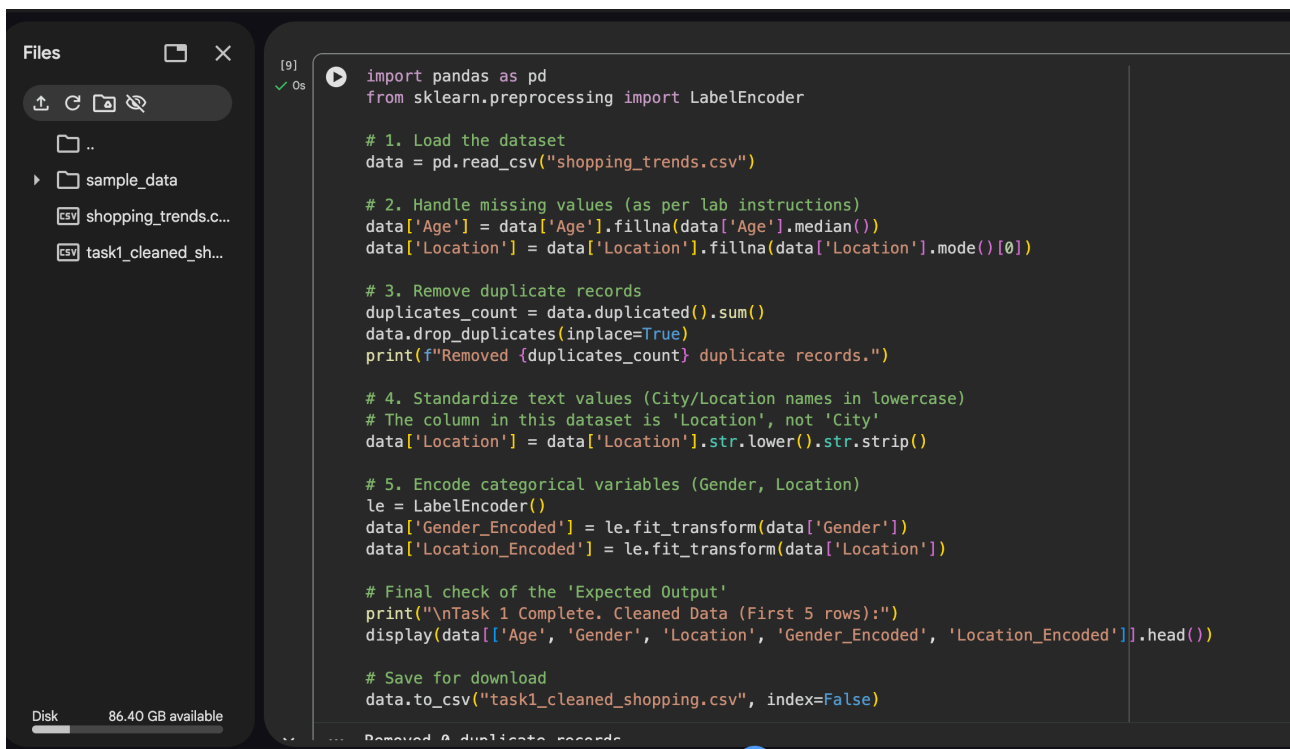
Batch-08

Task 1 – Customer Data Cleaning

Prompt:

Generate Python code using pandas to:

- Handle missing values in age, income, city
- Remove duplicates
- Convert city names to lowercase
- Encode gender and city columns



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with files like 'sample_data', 'shopping_trends.csv', and 'task1_cleaned_sh...'. The code editor contains the following Python code:

```
[9]
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# 1. Load the dataset
data = pd.read_csv("shopping_trends.csv")

# 2. Handle missing values (as per lab instructions)
data['Age'] = data['Age'].fillna(data['Age'].median())
data['Location'] = data['Location'].fillna(data['Location'].mode()[0])

# 3. Remove duplicate records
duplicates_count = data.duplicated().sum()
data.drop_duplicates(inplace=True)
print(f"Removed {duplicates_count} duplicate records.")

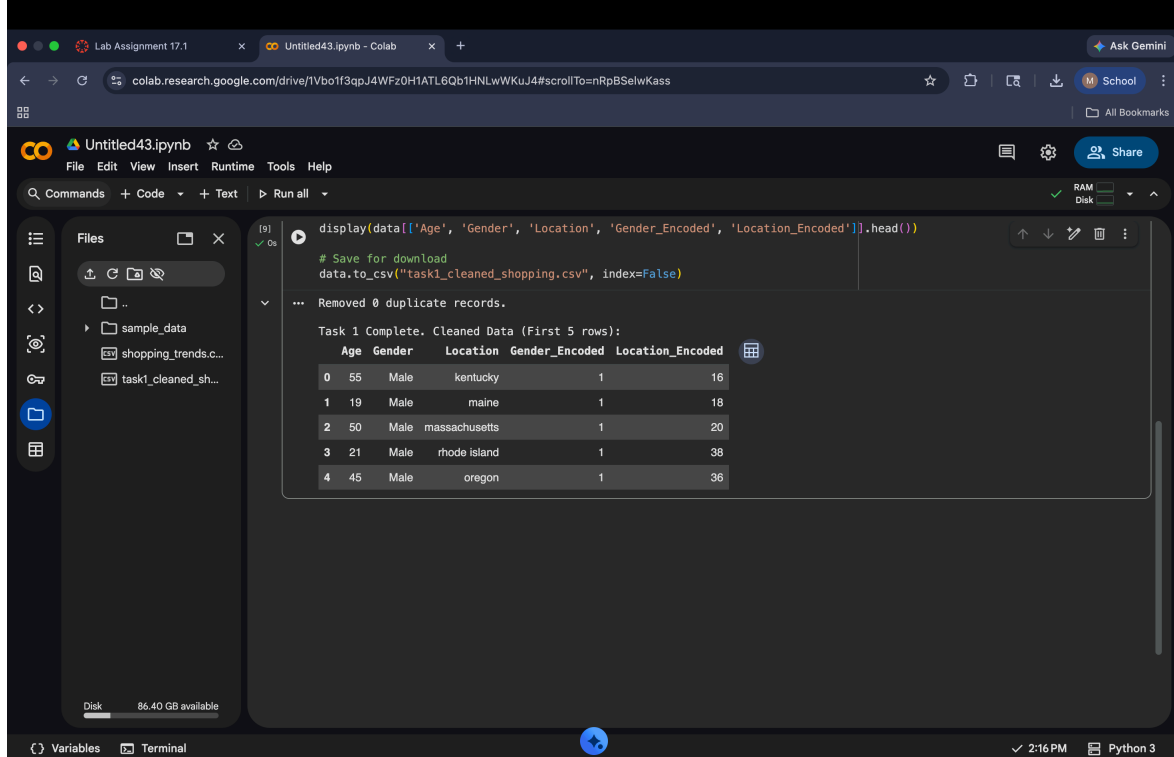
# 4. Standardize text values (City/Location names in lowercase)
# The column in this dataset is 'Location', not 'City'
data['Location'] = data['Location'].str.lower().str.strip()

# 5. Encode categorical variables (Gender, Location)
le = LabelEncoder()
data['Gender_Encoded'] = le.fit_transform(data['Gender'])
data['Location_Encoded'] = le.fit_transform(data['Location'])

# Final check of the 'Expected Output'
print("\nTask 1 Complete. Cleaned Data (First 5 rows):")
display(data[['Age', 'Gender', 'Location', 'Gender_Encoded', 'Location_Encoded']].head())

# Save for download
data.to_csv("task1_cleaned_shopping.csv", index=False)
```

Below the code editor, a status bar indicates "Disk 86.40 GB available". At the bottom of the notebook, a message states "Removed 0 duplicate records".

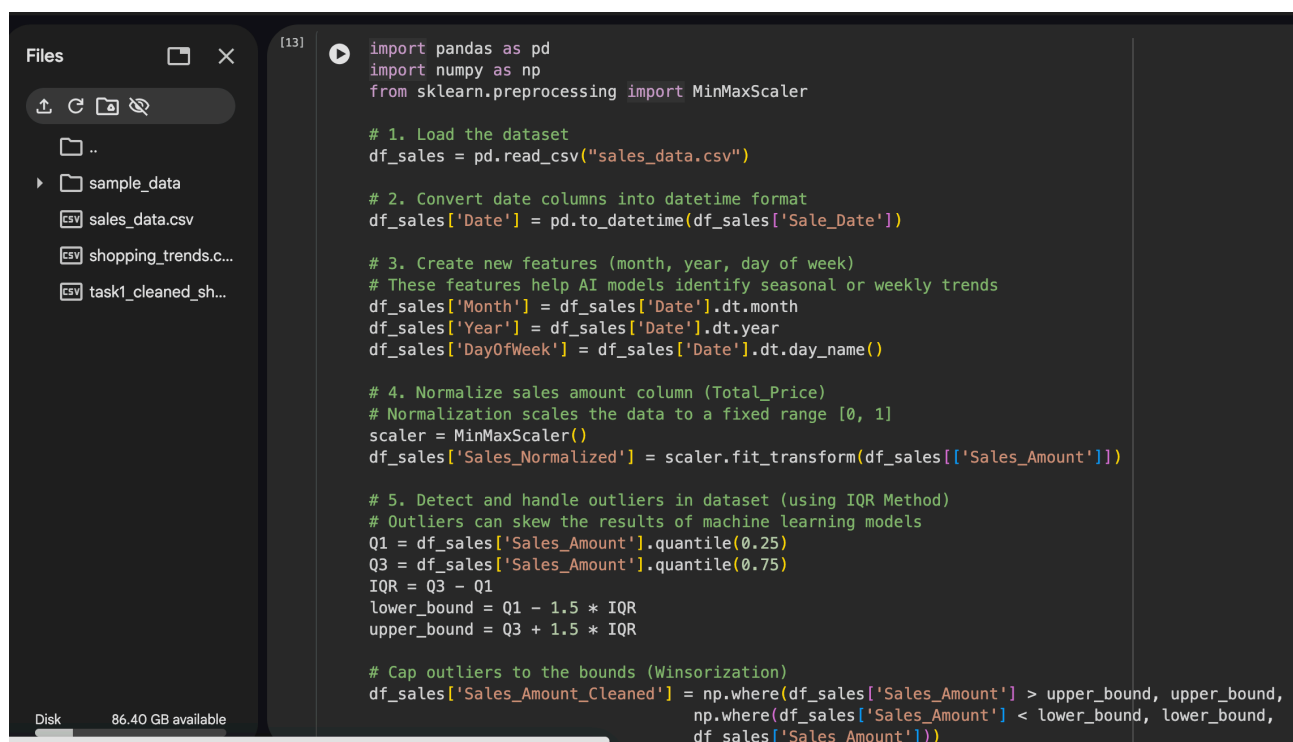


Task 2 – Sales Data Preprocessing

Prompt:

Write Python code to:

- Convert date column to datetime
- Extract month, year, day of week
- Normalize sales column
- Detect and remove outliers using IQR



Task 2 Complete. Transformed Dataset Sample:

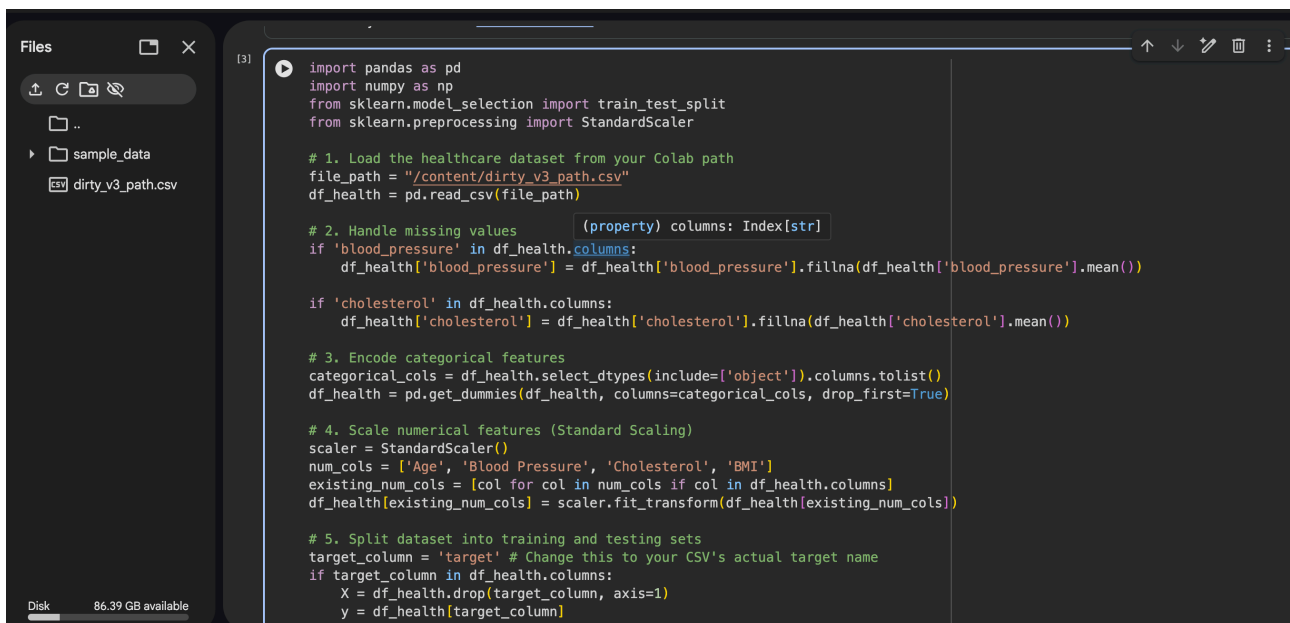
	Date	Month	DayOfWeek	Sales_Amount	Sales_Normalized
0	2023-02-03	2	Friday	5053.97	0.500950
1	2023-04-21	4	Friday	4384.02	0.433202
2	2023-09-21	9	Thursday	4631.23	0.458201
3	2023-08-24	8	Thursday	2167.94	0.209105
4	2023-03-24	3	Friday	3750.20	0.369108

Task 3 – Healthcare Data Preparation:

Prompt;

Generate Python code to:

- Fill missing values in blood_pressure and cholesterol
- Encode categorical variables
- Scale numerical features
- Split dataset into train and test sets



```
[3] import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load the healthcare dataset from your Colab path
file_path = "/content/dirty_v3_path.csv"
df_health = pd.read_csv(file_path)

# 2. Handle missing values
df_health['blood_pressure'] = df_health['blood_pressure'].fillna(df_health['blood_pressure'].mean())

if 'cholesterol' in df_health.columns:
    df_health['cholesterol'] = df_health['cholesterol'].fillna(df_health['cholesterol'].mean())

# 3. Encode categorical features
categorical_cols = df_health.select_dtypes(include=['object']).columns.tolist()
df_health = pd.get_dummies(df_health, columns=categorical_cols, drop_first=True)

# 4. Scale numerical features (Standard Scaling)
scaler = StandardScaler()
num_cols = ['Age', 'Blood Pressure', 'Cholesterol', 'BMI']
existing_num_cols = [col for col in num_cols if col in df_health.columns]
df_health[existing_num_cols] = scaler.fit_transform(df_health[existing_num_cols])

# 5. Split dataset into training and testing sets
target_column = 'target' # Change this to your CSV's actual target name
if target_column in df_health.columns:
    X = df_health.drop(target_column, axis=1)
    y = df_health[target_column]
```

Files

📁

🔍

📄

🔒

sample_data

dirty_v3_path.csv

[3]

▶

```
if target_column in df_health.columns:
    X = df_health.drop(target_column, axis=1)
    y = df_health[target_column]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    print("Task 3 Complete.")
    print(f"Training features shape: {X_train.shape}")
    print(f"Testing features shape: {X_test.shape}")
else:
    print(f"Task 3 Processed, but target column '{target_column}' not found for splitting.")

# Display preprocessed data
display(df_health.head())

# Save the preprocessed dataset
df_health.to_csv("/content/preprocessed_healthcare_data.csv", index=False)
```

↑

↓

🔗

🗑️

⋮

... Task 3 Processed, but target column 'target' not found for splitting.

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	...	Gender_Male
0	-0.539449	137.04	-0.246013	0.073986	96.04	6	0.562248	210.56	7.61	0	...	True
1	-2.041956	71.58	-1.289779	-0.381614	97.54	2	-1.416021	129.41	4.91	0	...	True
2	-0.289031	95.24	NaN	-1.037957	90.31	2	0.056866	165.35	5.60	0	...	False
3	0.149200	NaN	-0.470897	1.744520	96.60	5	-0.457168	182.13	6.92	0	...	False
4	0.712640	95.15	1.789331	0.461508	94.90	4	1.387149	115.85	5.98	0	...	False

5 rows x 27 columns

...

	Medical Condition_Asthma	Medical Condition_Cancer	Medical Condition_Diabetes	Medical Condition_Healthy	Medical Condition_Hypertension	Medical Condition_Obesity	random_
	False	False	True	False	False	False	
	False	False	False	True	False	False	
	True	False	False	False	False	False	
	False	False	False	False	False	True	
	False	False	False	False	True	False	

...

l s	Medical Condition_Healthy	Medical Condition_Hypertension	Medical Condition_Obesity	random_notes_??	random_notes_ipsum	random_notes_lorem
e	False	False	False	False	False	True
e	True	False	False	False	True	False
e	False	False	False	False	True	False
e	False	False	True	False	False	True
e	False	True	False	False	False	True

Task 4 – Employee Salary Data Preprocessing

Prompt:

Write Python code to:

- Fill missing values
- Remove salary outliers using IQR
- Convert department names to lowercase
- Encode job location and department

```
[16] ▶ import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from scipy import stats

# 1. Load the dataset
file_path = "/content/Employee_Salary_Dataset.csv" # Changed from dirty_v3_path.csv to Employee_Salary_Dataset.csv
df_salary = pd.read_csv(file_path)

# 2. Handle missing values
if 'salary' in df_salary.columns:
    df_salary['salary'] = df_salary['salary'].fillna(df_salary['salary'].median())

# Fill categorical gaps with 'unknown'
df_salary = df_salary.fillna("unknown")

# 3. Detect and remove salary outliers using Z-score
if 'salary' in df_salary.columns:
    # Calculate absolute Z-scores
    z_scores = np.abs(stats.zscore(df_salary['salary']))
    # Keep only data within 3 standard deviations
    df_salary = df_salary[z_scores < 3]
    print(f"Outliers removed. Cleaned dataset rows: {len(df_salary)}")

# 4. Standardize department names
if 'department' in df_salary.columns:
    df_salary['department'] = df_salary['department'].astype(str).str.lower().str.strip()

# 5. Apply Label Encoding to job location and department
le = LabelEncoder()

if 'job_location' in df_salary.columns:
```

```
[16] ▶ print(f"Outliers removed. Cleaned dataset rows: {len(df_salary)}")

# 4. Standardize department names
if 'department' in df_salary.columns:
    df_salary['department'] = df_salary['department'].astype(str).str.lower().str.strip()

# 5. Apply Label Encoding to job location and department
le = LabelEncoder()

if 'job_location' in df_salary.columns:
    df_salary['location_encoded'] = le.fit_transform(df_salary['job_location'])

if 'department' in df_salary.columns:
    df_salary['dept_encoded'] = le.fit_transform(df_salary['department'])

print("\nTask 4 Complete. Structured HR Analytics Data (First 5 rows):")
display(df_salary.head())

df_salary.to_csv("/content/structured_hr_analytics.csv", index=False)
```

df_salary.to_csv('content/structured_hr_analytics.csv', index=False)

...

Task 4 Complete. Structured HR Analytics Data (First 5 rows):

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000